

✓ Loading and Displaying 'lion.jpg'

Now, let's load `/content/lion.jpg` and display it. We'll again convert it from BGR to RGB for correct `matplotlib` display.

```
import cv2
import matplotlib.pyplot as plt

# Define the path to the new image
lion_image_path = '/content/lion.jpg'

# Load the image using OpenCV
lion_original_image = cv2.imread(lion_image_path)

# Check if the image was loaded successfully
if lion_original_image is None:
    print(f"Error: Could not load image from {lion_image_path}. Please che
else:
    # Convert the image from BGR to RGB for correct display with matplotlib
    lion_rgb_image = cv2.cvtColor(lion_original_image, cv2.COLOR_BGR2RGB)

    # Display the image
    plt.figure(figsize=(10, 10))
    plt.imshow(lion_rgb_image)
    plt.title(f'Loaded Image: {lion_image_path}')
    plt.axis('off')
    plt.show()
```

Loaded Image: /content/lion.jpg



✓ Applying Sobel Operator to the 'lion.jpg' Image

Now we'll convert the loaded 'lion.jpg' image to grayscale and then apply the Sobel operator to compute the gradient in both x and y directions. We'll ensure the image is converted to a floating-point type before applying Sobel to get accurate results.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Assuming 'lion_original_image' is loaded from the previous step
if 'lion_original_image' not in locals() or lion_original_image is None:
    print("Error: 'lion_original_image' not found or not loaded. Please ensu
else:
    # Convert the image to grayscale, as Sobel typically operates on single-
    gray_image = cv2.cvtColor(lion_original_image, cv2.COLOR_BGR2GRAY)

    # Convert image to float32 for Sobel to ensure correct gradient calculat
    gray_image_float = np.float32(gray_image)

    # Apply Sobel operator in X direction
    # ddepth=cv2.CV_64F for output to accurately store gradient values (incl
    sobelx_lion = cv2.Sobel(gray_image_float, cv2.CV_64F, 1, 0, ksize=3)

    # Apply Sobel operator in Y direction
    sobely_lion = cv2.Sobel(gray_image_float, cv2.CV_64F, 0, 1, ksize=3)

    print(f"SobelX (Lion) min: {sobelx_lion.min():.2f}, max: {sobelx_lion.ma
    print(f"SobelY (Lion) min: {sobely_lion.min():.2f}, max: {sobely_lion.ma

    # Display Sobel X and Y results
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 3, 1)
    plt.imshow(gray_image, cmap='gray')
    plt.title('Grayscale Lion Image')
    plt.axis('off')

    plt.subplot(1, 3, 2)
    plt.imshow(sobelx_lion, cmap='gray', vmin=sobelx_lion.min(), vmax=sobelx
    plt.title('Sobel X (Gx) for Lion')
    plt.axis('off')

    plt.subplot(1, 3, 3)
    plt.imshow(sobely_lion, cmap='gray', vmin=sobely_lion.min(), vmax=sobely
    plt.title('Sobel Y (Gy) for Lion')
    plt.axis('off')

    plt.show()
```

SobelX (Lion) min: -577.00, max: 543.00

SobelY (Lion) min: -577.00, max: 598.00

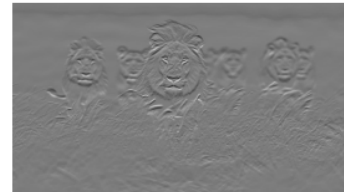
Grayscale Lion Image



Sobel X (Gx) for Lion



Sobel Y (Gy) for Lion



✓ Calculating Gradient Magnitude and Direction for 'lion.jpg'

Now, let's compute and visualize the gradient magnitude and direction for the 'lion.jpg' image using the `sobelx_lion` and `sobely_lion` results.

```
# Calculate the gradient magnitude for the lion image
gradient_magnitude_lion = np.sqrt(sobelx_lion**2 + sobely_lion**2)

# Normalize for display
gradient_magnitude_display_lion = cv2.normalize(gradient_magnitude_lion, None, 0, 255, cv2.NORM_MINMAX)

# Calculate the gradient direction (angle in radians)
gradient_direction_lion = np.arctan2(sobely_lion, sobelx_lion)

# Convert direction to degrees
gradient_direction_degrees_lion = np.degrees(gradient_direction_lion)

# Adjust angles to be in the range [0, 360) and normalize for display
gradient_direction_display_lion = (gradient_direction_degrees_lion + 360) % 360
gradient_direction_display_lion = cv2.normalize(gradient_direction_display_lion, None, 0, 255, cv2.NORM_MINMAX)

print(f"Gradient Magnitude (Lion) min: {gradient_magnitude_lion.min():.2f}, max: {gradient_magnitude_lion.max():.2f}")
print(f"Gradient Direction (Lion, degrees) min: {gradient_direction_degrees_lion.min():.2f}, max: {gradient_direction_degrees_lion.max():.2f}")

# Display gradient magnitude and direction
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(gradient_magnitude_display_lion, cmap='gray')
plt.title('Gradient Magnitude (Lion)')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(gradient_direction_display_lion, cmap='hsv')
plt.title('Gradient Direction (Lion, Mapped 0-255)')
plt.axis('off')

plt.show()

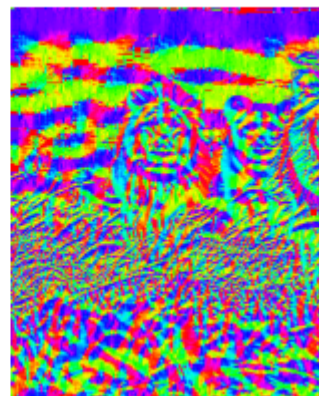
print("Gradient magnitude and direction for 'lion.jpg' calculated and displayed")
```

Gradient Magnitude (Lion) min: 0.00, max: 630.63
 Gradient Direction (Lion, degrees) min: -179.87, max: 180.00

Gradient Magnitude (Lion)



Gradient Direction



Gradient magnitude and direction for 'lion.jpg' calculated and displayed.

✓ Applying the Prewitt Operator to 'lion.jpg'

The Prewitt operator is a different set of convolution kernels designed for edge detection. We will define the horizontal (Gx) and vertical (Gy) kernels and apply them using `cv2.filter2D`. We'll then compute the gradient magnitude and direction for Prewitt.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Ensure gray_image_float is available from previous steps
if 'gray_image_float' not in locals():
    print("Error: 'gray_image_float' not found. Please run the previous cell")
else:
    # Define Prewitt kernels
    kernelx_prewitt = np.array([[ -1,  0,  1],
                                [ -1,  0,  1],
                                [ -1,  0,  1]], dtype=np.float32)

    kernely_prewitt = np.array([[ -1, -1, -1],
                                [  0,  0,  0],
                                [  1,  1,  1]], dtype=np.float32)

    # Apply Prewitt operator in X direction
    Gx_prewitt = cv2.filter2D(gray_image_float, -1, kernelx_prewitt)

    # Apply Prewitt operator in Y direction
    Gy_prewitt = cv2.filter2D(gray_image_float, -1, kernely_prewitt)

    print(f"Prewitt Gx min: {Gx_prewitt.min():.2f}, max: {Gx_prewitt.max():.2f}")
    print(f"Prewitt Gy min: {Gy_prewitt.min():.2f}, max: {Gy_prewitt.max():.2f}")

    # Display Prewitt Gx and Gy results
```

```
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.imshow(Gx_prewitt, cmap='gray', vmin=Gx_prewitt.min(), vmax=Gx_prewi
plt.title('Prewitt Gx for Lion')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(Gy_prewitt, cmap='gray', vmin=Gy_prewitt.min(), vmax=Gy_prewi
plt.title('Prewitt Gy for Lion')
plt.axis('off')

plt.show()
```

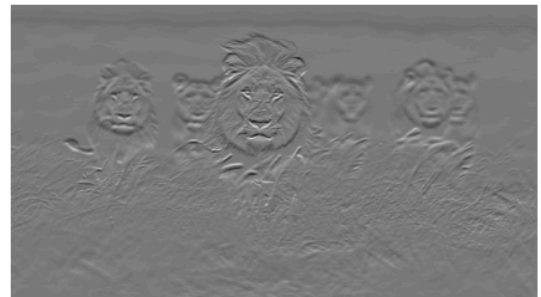
Prewitt Gx min: -425.00, max: 401.00

Prewitt Gy min: -426.00, max: 449.00

Prewitt Gx for Lion



Prewitt Gy for Lion



✓ Calculating Prewitt Gradient Magnitude and Direction

Now, let's compute and visualize the gradient magnitude and direction using the Prewitt **Gx** and **Gy** results.

```
import numpy as np
import matplotlib.pyplot as plt

# Assuming Gx_prewitt and Gy_prewitt are available from the previous step
if 'Gx_prewitt' not in locals() or 'Gy_prewitt' not in locals():
    print("Error: Prewitt Gx and Gy results not found. Please run the previo
else:
    # Calculate the gradient magnitude for Prewitt
    gradient_magnitude_prewitt = np.sqrt(Gx_prewitt**2 + Gy_prewitt**2)

    # Normalize for display
    gradient_magnitude_display_prewitt = cv2.normalize(gradient_magnitude_pr

    # Calculate the gradient direction (angle in radians) for Prewitt
    gradient_direction_prewitt = np.arctan2(Gy_prewitt, Gx_prewitt)

    # Convert direction to degrees
    gradient_direction_degrees_prewitt = np.degrees(gradient_direction_prewi

    # Adjust angles to be in the range [0, 360) and normalize for display
    gradient_direction_display_prewitt = (gradient_direction_degrees_prewitt
```



```

gradient_direction_display_prewitt = cv2.normalize(gradient_direction_di

print(f"Prewitt Gradient Magnitude min: {gradient_magnitude_prewitt.min(
print(f"Prewitt Gradient Direction (degrees) min: {gradient_direction_de

# Display gradient magnitude and direction
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(gradient_magnitude_display_prewitt, cmap='gray')
plt.title('Prewitt Gradient Magnitude')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(gradient_direction_display_prewitt, cmap='hsv')
plt.title('Prewitt Gradient Direction (Mapped 0-255)')
plt.axis('off')

plt.show()

print("Prewitt gradient magnitude and direction calculated and displayed

```

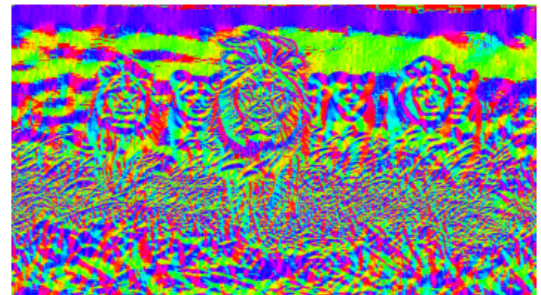
Prewitt Gradient Magnitude min: 0.00, max: 456.97

Prewitt Gradient Direction (degrees) min: -179.64, max: 180.00

Prewitt Gradient Magnitude



Prewitt Gradient Direction (Mapped 0-255)



Prewitt gradient magnitude and direction calculated and displayed.

✓ Applying the Robert Cross Operator to 'lion.jpg'

The Robert Cross operator uses two 2x2 convolution kernels to detect edges, focusing on diagonal differences. We will define these kernels and apply them using `cv2.filter2D`, then compute the gradient magnitude and direction.

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

# Ensure gray_image_float is available from previous steps
if 'gray_image_float' not in locals():
    print("Error: 'gray_image_float' not found. Please run the previous cell
else:
    # Define Robert Cross kernels (2x2)
    kernelx_roberts = np.array([[1, 0],
                                [0, -1]], dtype=np.float32)

```

```

kernelx_roberts = np.array([[0, 1],
                             [-1, 0]], dtype=np.float32)

# Apply Robert Cross operator in X direction
# Note: filter2D will handle the kernel size, but the output image will
Gx_roberts = cv2.filter2D(gray_image_float, -1, kernelx_roberts)

# Apply Robert Cross operator in Y direction
Gy_roberts = cv2.filter2D(gray_image_float, -1, kernely_roberts)

print(f"Robert Cross Gx min: {Gx_roberts.min():.2f}, max: {Gx_roberts.ma
print(f"Robert Cross Gy min: {Gy_roberts.min():.2f}, max: {Gy_roberts.ma

# Display Robert Cross Gx and Gy results
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.imshow(Gx_roberts, cmap='gray', vmin=Gx_roberts.min(), vmax=Gx_rober
plt.title('Robert Cross Gx for Lion')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(Gy_roberts, cmap='gray', vmin=Gy_roberts.min(), vmax=Gy_rober
plt.title('Robert Cross Gy for Lion')
plt.axis('off')

plt.show()

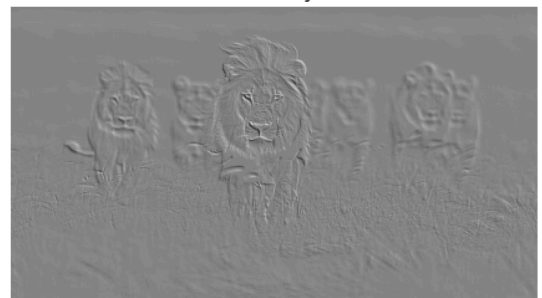
```

Robert Cross Gx min: -149.00, max: 131.00
 Robert Cross Gy min: -152.00, max: 137.00

Robert Cross Gx for Lion



Robert Cross Gy for Lion



✓ Calculating Robert Cross Gradient Magnitude and Direction

Now, let's compute and visualize the gradient magnitude and direction using the Robert Cross **Gx** and **Gy** results.

```

import numpy as np
import matplotlib.pyplot as plt

# Assuming Gx_roberts and Gy_roberts are available from the previous step
if 'Gx_roberts' not in locals() or 'Gy_roberts' not in locals():
    print("Error: Robert Cross Gx and Gy results not found. Please run the p

```

```

else:
    # Calculate the gradient magnitude for Robert Cross
    gradient_magnitude_roberts = np.sqrt(Gx_roberts**2 + Gy_roberts**2)

    # Normalize for display
    gradient_magnitude_display_roberts = cv2.normalize(gradient_magnitude_ro

    # Calculate the gradient direction (angle in radians) for Robert Cross
    gradient_direction_roberts = np.arctan2(Gy_roberts, Gx_roberts)

    # Convert direction to degrees
    gradient_direction_degrees_roberts = np.degrees(gradient_direction_rober

    # Adjust angles to be in the range [0, 360) and normalize for display
    gradient_direction_display_roberts = (gradient_direction_degrees_roberts
    gradient_direction_display_roberts = cv2.normalize(gradient_direction_di

    print(f"Robert Cross Gradient Magnitude min: {gradient_magnitude_roberts
    print(f"Robert Cross Gradient Direction (degrees) min: {gradient_directi

    # Display gradient magnitude and direction
    plt.figure(figsize=(10, 5))
    plt.subplot(1, 2, 1)
    plt.imshow(gradient_magnitude_display_roberts, cmap='gray')
    plt.title('Robert Cross Gradient Magnitude')
    plt.axis('off')

    plt.subplot(1, 2, 2)
    plt.imshow(gradient_direction_display_roberts, cmap='hsv')
    plt.title('Robert Cross Gradient Direction (Mapped 0-255)')
    plt.axis('off')

    plt.show()

    print("Robert Cross gradient magnitude and direction calculated and disp

```

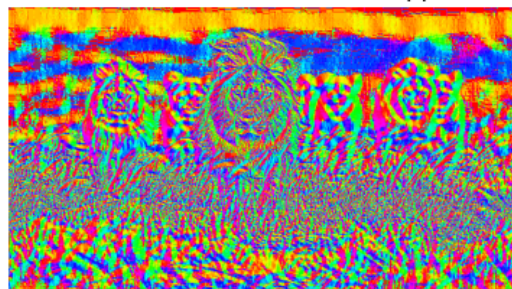
Robert Cross Gradient Magnitude min: 0.00, max: 170.24

Robert Cross Gradient Direction (degrees) min: -179.53, max: 180.00

Robert Cross Gradient Magnitude



Robert Cross Gradient Direction (Mapped 0-255)



Robert Cross gradient magnitude and direction calculated and displayed.

